

# CS633 Assignment Group Number 21

Aryan Verma (220230)  
Ayush Kumar Yadav (220267)  
Mohd Mufeed Amir (220660)  
Nishant Pandey (220724)  
Sankalp Mittal (220963)

February 15, 2026

## 1 Code Description

The program implements a parallel data exchange pipeline using the Message Passing Interface (MPI). The core logic simulates a dual-dependency system where each process (rank) interacts with neighbors at distances  $D1$  and  $D2$ .

### 1.1 Communication Strategy: Odd-Even Ordering

To prevent deadlock and avoid the serialization of communication while using blocking `MPI_Send` and `MPI_Recv` calls, an **Odd-Even (Red-Black) ordering** strategy is employed.

- **Parity Determination:** Processes are logically grouped into blocks based on their distance from the source. For a dependency  $D$ , the parity is determined by the expression  $(rank/D)\%2$ .
- **Conflict Avoidance:** Ranks with even parity execute an `MPI_Send` followed by an `MPI_Recv`, while ranks with odd parity execute an `MPI_Recv` followed by an `MPI_Send`. This ensures that for every blocked sender, there is a corresponding rank ready to receive, maintaining pipeline fluidity.
- **Running Time Analysis:** This code will execute in 2 communication rounds each time `doCommunication` is called and since this is called 4 times. There are 8 communication rounds regardless of the number of processes

```
1 // if cases of 'only sending' and 'only receiving' ranks
2 ...
3 else if (i_am_sender && i_am_receiver)
4 {
5     // Middle: Parity Check
6     if ((rank / d) % 2 != 0)
```

```

7      { // Odd parity block
8          if(is_forward){
9              MPI_Recv(recv_buf, M, MPI_DOUBLE, source, tag,
10                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
11              MPI_Send(data_to_send, M, MPI_DOUBLE, target, tag,
12                 MPI_COMM_WORLD);
13          }
14          else{
15              MPI_Send(data_to_send, M, MPI_DOUBLE, source, tag,
16                 MPI_COMM_WORLD);
17              MPI_Recv(recv_buf, M, MPI_DOUBLE, target, tag,
18                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
19          }
20      }
21      else
22      { // Even parity block
23          if(is_forward){
24              MPI_Send(data_to_send, M, MPI_DOUBLE, target, tag,
25                 MPI_COMM_WORLD);
26              MPI_Recv(recv_buf, M, MPI_DOUBLE, source, tag,
27                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
28          }
29          else{
30              MPI_Recv(recv_buf, M, MPI_DOUBLE, target, tag,
31                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
32              MPI_Send(data_to_send, M, MPI_DOUBLE, source, tag,
33                 MPI_COMM_WORLD);
34          }
35      }
36  }
37  ...

```

Listing 1: Odd-Even Ordering in doCommunication Function

## 1.2 Pipeline Phases

The simulation executes for  $T$  iterations, with each iteration divided into four distinct phases:

1. **Forward Communication:** Valid senders transmit  $M$  doubles to ranks at  $rank + D1$  and  $rank + D2$ .

2. **Computation at Receivers:**

- **D1 Receivers:** Perform element-wise squaring:

$$\text{data\_at\_D1}[i] = \text{data\_received}[i] * \text{data\_received}[i]$$

- **D2 Receivers:** Perform a logarithmic transformation:

$$\text{data\_at\_D2}[i] = \log(\text{data\_received}[i])$$

3. **Backward Communication:** Receivers return the processed results to the original senders using the same Odd-Even logic to prevent blocking.

To reuse code we use the same function but it takes an *is\_forward* flag to handle the required changes.

4. **Sender Buffer Update:** Upon receiving results, senders update their data for the next iteration:

- **D1 Update:**

```
data_to_send_D1[i] = (unsigned long long)data_recvd_from_D1[i] % 100000
```

.

- **D2 Update:**

```
data_to_send_D2[i] = data_recvd_from_D2[i] × 100000
```

.

### 1.3 Global Reduction and Analysis

After  $T$  iterations, each valid sender identifies the local maximum value for both D1 and D2 datasets. A manual **Tree-based Reduction** is implemented to find the global maximum across all processors:

- The algorithm uses a bitwise XOR approach ( $rank \oplus step$ ) to identify communication partners at each level of the tree, resulting in  $O(\log P)$  complexity.
- Rank 0 aggregates these values and prints the final global maximums for both  $D1$  and  $D2$ .
- The total execution time (spanning  $T$  iterations and the final reduction) is collected using `MPI_Reduce` with the `MPI_MAX` operator at Rank 0.
- This code does  $\log_2 P$  rounds of communication so this component goes up as the number of processes increases

```

1 double global_max_D1 = local_max_D1;
2 double global_max_D2 = local_max_D2;
3
4 int step = 1;
5 while (step < P)
6 {
7     int partner = rank ^ step; // XOR finds the neighbor in the
8     // current tree level
9     if (partner < P)
10    {
11        if (rank < partner)
12        {
13            // Low rank receives and aggregates
14            double rcv_val_D1, rcv_val_D2;
```

```

15     MPI_Recv(&recv_val_D1, 1, MPI_DOUBLE, partner,
16     TAG_RED_D1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
17     MPI_Recv(&recv_val_D2, 1, MPI_DOUBLE, partner,
18     TAG_RED_D2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
19
20     global_max_D1 = fmax(global_max_D1, recv_val_D1);
21     global_max_D2 = fmax(global_max_D2, recv_val_D2);
22 }
23 else
24 {
25     // High rank sends and retires from the reduction
26     MPI_Send(&global_max_D1, 1, MPI_DOUBLE, partner,
27     TAG_RED_D1, MPI_COMM_WORLD);
28     MPI_Send(&global_max_D2, 1, MPI_DOUBLE, partner,
29     TAG_RED_D2, MPI_COMM_WORLD);
30     break;
31 }
32 }
33 step <= 1; // Move to next level of tree
34 }

```

Listing 2: Bitwise Tree Reduction for Global Maxima

## 2 Code Compilation and Execution Instructions

The implementation is designed to run on the Param Rudra HPC cluster using the SLURM workload manager. The following steps outline the procedure for environment setup, compilation, and job submission.

### 2.1 Environment Preparation

Before compilation, a clean environment must be established to prevent dependency conflicts between different compiler versions. The following modules are required:

- compiler/gcc/12.3
- compiler/openmpi/4.1.5

The environment is initialized using the command:

```
module purge && module load compiler/gcc/12.3 compiler/openmpi/4.1.5
```

### 2.2 Compilation

The source code `src.c` is compiled using the `mpicc` wrapper. We utilize the `-O3` optimization flag for enhanced performance and the `-lm` flag to link the standard math library required for `log()` computations.

```
mpicc src.c -o src_exec -lm -O3
```

## 2.3 Job Submission and Execution

The experiment requires execution across three process configurations

$$P = 8, 16, 32$$

Each configuration is handled by a specific SLURM batch script that allocates the necessary CPU resources and defines the execution parameters

$$M, D1, D2, T, \text{seed}$$

The following script manages the allocation of 8 cores on a single node and executes the simulation for both  $M$  values across 5 iterations.

```
1  #!/bin/bash
2
3  #SBATCH --job-name=bench_P8
4  #SBATCH -N 1 # Request 1 Node
5  #SBATCH --ntasks-per-node=8 # Allocate half node (8
   cores)
6  #SBATCH --output=results_P8_%j.out # Save output to specific
   P8 file
7  #SBATCH --error=results_P8_%j.err
8  #SBATCH --partition=cpu
9  #SBATCH --time=00:10:00
10
11 # --- Configuration ---
12 P=8
13 D1=2
14 D2=4
15 T=10
16 SEED=1000
17 M1=262144
18 M2=1048576
19
20 echo "===== "
21 echo "Starting Experiment for P=$P"
22 echo "Node Allocation: $SLURM_JOB_NUM_NODES nodes"
23 echo "===== "
24
25 module purge
26 module load compiler/gcc/12.3
27 module load compiler/openmpi/4.1.5
28
29 # --- Execution Loop (Repeat 5 times) ---
30 for i in {1..5}
31 do
32     echo "Iteration $i of 5..."
33
34     # Configuration 1: M = 262,144
35     echo " [P=$P] [M=$M1] Running..."
```

```

36 mpirun -np $P ./src_exec $M1 $D1 $D2 $T $SEED
37
38 # Configuration 2: M = 1,048,576
39 echo " [P=$P][M=$M2] Running..."
40 mpirun -np $P ./src_exec $M2 $D1 $D2 $T $SEED
41 done
42
43 echo "Experiment P=$P Complete."

```

Listing 3: SLURM Job Script for 8 Processes

The shell script for 16 and 32 processes follow similarly.

To replicate the results, the jobs are submitted as follows:

```

sbatch run_P8.sh
sbatch run_P16.sh
sbatch run_P32.sh

```

### 3 Results

Upon successful completion, the simulation outputs the global maximums and execution times to standard output files named `results_P<process>_n<run>.out`. For each process configuration ( $P = 8, 16, 32$ ), we conducted five independent runs, resulting in five output files per process count. Each file contains ten execution time values—five recorded for each value of  $M$ . These data points were then aggregated to compute two average execution times (one for each value of  $M$ ) per file. These resulting averages served as the primary data points for our subsequent analysis and plotting.

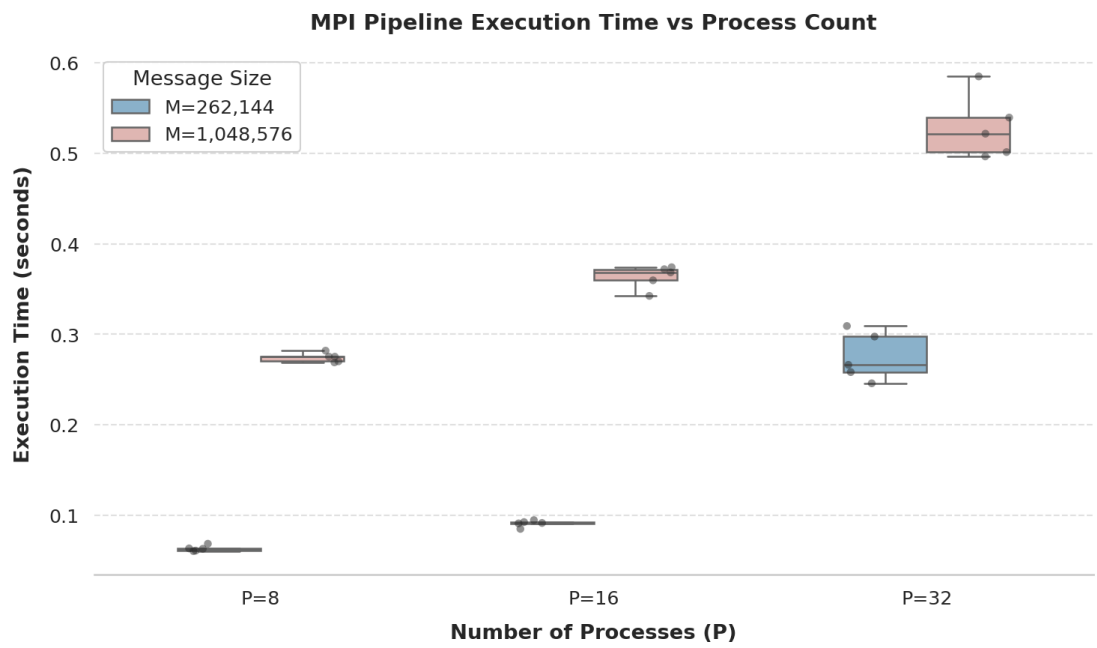
Table 1: Complete Execution Times (in seconds) for all configurations

| Process Count (P) | Message Size (M) | It 1     | It 2     | It 3     | It 4     | It 5     | Average         |
|-------------------|------------------|----------|----------|----------|----------|----------|-----------------|
| 8                 | 262,144          | 0.060277 | 0.068422 | 0.062839 | 0.063276 | 0.060804 | <b>0.063124</b> |
| 8                 | 1,048,576        | 0.268953 | 0.281943 | 0.275317 | 0.275278 | 0.270129 | <b>0.274324</b> |
| 16                | 262,144          | 0.090931 | 0.092373 | 0.091415 | 0.084849 | 0.094549 | <b>0.090823</b> |
| 16                | 1,048,576        | 0.359709 | 0.368446 | 0.371990 | 0.342490 | 0.374188 | <b>0.363365</b> |
| 32                | 262,144          | 0.258224 | 0.297482 | 0.266257 | 0.245825 | 0.309165 | <b>0.275391</b> |
| 32                | 1,048,576        | 0.501498 | 0.585146 | 0.496710 | 0.521844 | 0.539692 | <b>0.528978</b> |

### 4 Conclusions

Across the different runs we executed and gradually changed our code we noticed the following

- As expected the running time increases as the message size and process count increases



- The rate of change of time with process increases much rapidly as the message size increases
- The algorithm only shows significant improvement in times from the trivial algorithm for high number of processes as the constant terms in the algorithmic complexity become less relevant